

0.83.1. Un son quand on ramasse : note — et un bruit contre le mur

Le labyrinthe du 0.82.1, plus un bruit quand on se cogne : `bruit(4, 8)`, sur la voix du bruit.

1. Ce qu'il faut comprendre

C'est le labyrinthe du 0.82.1, plus un son quand le A se **cogne** contre un mur.

Ce qui est nouveau ici : `bruit(duree, volume)`. Elle frappe sur la **voix du bruit** (la 4e) : pas une note, un « toc ».
». `bruit(4, 8)` : 4 images, volume 8.

Où le mettre : dans le `else` du test du mur. Si la case d'arrivée **n'est pas** un M, on avance ; **sinon** (`else`), c'en est un : on reste, et on fait « toc ».

Et une note quand on ramasse la pièce, comme au 0.83.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2320 octets** de cartouche.

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- LES VARIABLES : les mêmes qu'au 0.82.1 ----
```

2. UNE VARIABLE — X

```
uint8_t x = 3;
```

3. UNE VARIABLE — Y

```
uint8_t y = 5;
```

4. UNE VARIABLE — NX

```
uint8_t nx = 0;
```

5. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

6. UNE VARIABLE — SCORE

```
uint8_t score = 0;
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ---- LE LABYRINTHE : tout en lettres M ----
    //
    // ligne 2 : M M M M M M M M M M M M M M M M M le haut
    // ligne 3 : M M M M M M M M M M M M M M M M M
    // ligne 5 : M A M P M A : départ, P : la pièce
    // ... M M M M M M M M M M M M M M M M M
    // ligne 10 : M M M M M M M M M M M M M M M M M le mur du milieu s'arrête ici...
    // ligne 11 : M M M M M M M M M M M M M M M M M ...les lignes 11 à 13
    // ligne 13 : M M M M M M M M M M M M M M M M M sont libres : on passe
    // ligne 14 : M M M M M M M M M M M M M M M M M le bas

    // Le haut et le bas : une ligne de 20 M, en UN seul texte.
    texte(0, 2, "MMMMMMMMMMMMMMMMMMMMMMMM");
    texte(0, 14, "MMMMMMMMMMMMMMMMMMMMMMMM");

    // Les deux côtés : un M par ligne, à gauche (colonne 0) et à droite (colonne 19).
    for (uint8_t l = 3; l < 14; l++) {
        poser(0, l, ALPHABET[12]);
        poser(19, l, ALPHABET[12]);
    }

    // Le mur du milieu, en colonne 10, des lignes 3 à 10 seulement : ouvert en bas.
    for (uint8_t l = 3; l <= 10; l++) {
        poser(10, l, ALPHABET[12]);
    }

    poser(16, 5, ALPHABET[15]); // la pièce P, de l'autre côté du mur
    poser(x, y, ALPHABET[0]); // le A, à son départ
    texte(0, 17, "SCORE");

    // ---- LA BOUCLE DU JEU ----
    while (true) {
        image();
        if (chaque(250)) { // au plus un pas tous les 250 ms (le 0.77)

            // ÉTAPE 1 : calculer la case d'ARRIVÉE, sans encore bouger.
            // On part de la case actuelle : si aucune flèche n'est tenue,
            // l'arrivée reste la case où l'on est.
            nx = x;
            ny = y;
            if (bouton(DROITE) && x < 19) { nx = x + 1; } // une colonne à droite
            if (bouton(GAUCHE) && x > 0) { nx = x - 1; } // une colonne à gauche
            if (bouton(BAS) && y < 16) { ny = y + 1; } // une ligne plus bas
            if (bouton(HAUT) && y > 0) { ny = y - 1; } // une ligne plus haut

            // ÉTAPE 2 : REGARDER ce qu'il y a sur la case d'arrivée, avec lire() :
            //
            // lire(nx, ny) != ALPHABET[12]
            // | |
            // | | +-- le M, la lettre des murs
            // | +----- != : « n'est pas »
            // +----- ce que montre l'écran à la case d'arrivée
            //
            // Exemple : le A est en (9, 8), la flèche droite est tenue.
            // L'arrivée est (10, 8). lire(10, 8) rend le M du mur :
            // « n'est pas un M » est FAUX → le A ne bouge pas.
            if (nx != x || ny != y) { // une flèche est tenue (|| : OU)
                if (lire(nx, ny) != ALPHABET[12]) {

                    // ÉTAPE 3 : ce n'est pas un mur, on avance.
                    effacer(x, y, 1); // efface le A à l'ancienne case
                    x = nx; // la case d'arrivée devient
                    y = ny; // la case du A
                }
            }
        }
    }
}
```

```

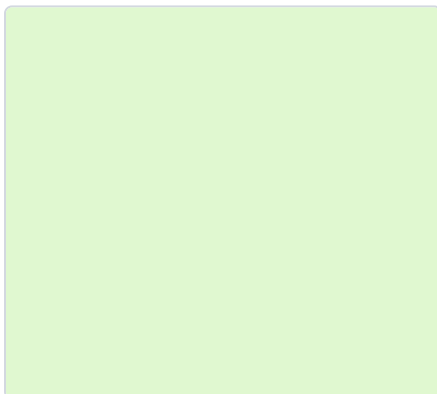
    poser(x, y, ALPHABET[0]);    // pose le A à la nouvelle case
} else {
    // ÉTAPE 3 bis : c'EST un mur. Le A ne bouge pas, et on fait « toc ».
    //
    //   bruit(4, 8);
    //       | |
    //       | +--- volume : 8 (de 0 à 15)
    //       +----- durée : 4 images, un bruit très court
    //
    // bruit joue sur la VOIX DU BRUIT, la 4e : pas une note, un choc.
    bruit(4, 8);
}
}
}

// La pièce : une seule fois (score == 0), avec un « ding » (le 0.83).
if (x == 16 && y == 5 && score == 0) {
    score = 1;
    note(1, D05, 10, 12);
}
nombre(6, 17, score);
}
}

```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



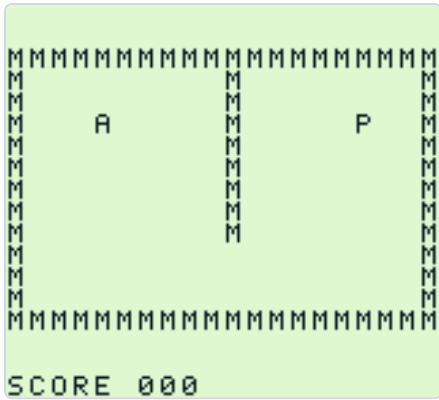
la toute première image — la console vient de démarrer — image 1



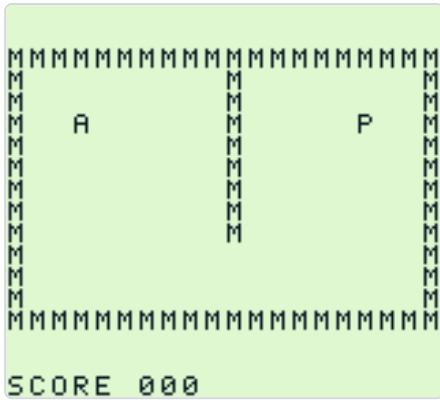
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



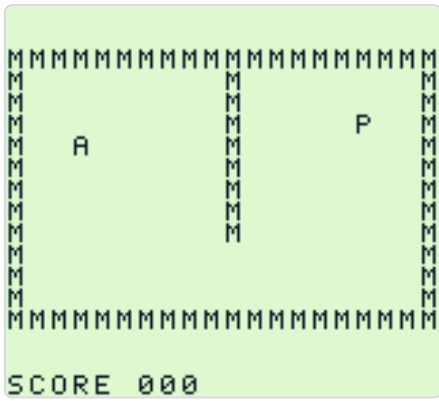
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran n'a pas bougé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le labyrinthe ; contre un mur, un « toc » ; sur la pièce, un « ding ».

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ contre le mur, la voix du bruit a frappé
- ✓ le A est resté devant le mur